

Validación de Símbolos de Agrupación

Ejercicio 1 · Estructuras de Datos con Pila

¿Qué se pide?

Dada una expresión matemática como texto, determinar si todos los símbolos de agrupación `()`, `[]` y `{}` están correctamente abiertos y cerrados, en el orden correcto.

Entrada: `"3 + [(2 * 5) - {8 / (4 - 2)}]"`

Salida esperada: `True`

¿Por qué se usa una Pila?

Una **pila (stack)** funciona con la regla *LIFO*: **L**ast **I**n, **F**irst **O**ut. El último símbolo abierto debe ser siempre el primero en cerrarse, exactamente igual que los paréntesis anidados.

Diagrama LIFO (Last In, First Out)

Acción	Símbolo	Estado de la Pila (tope ↑)	Resultado
Apertura	[	[← tope	Guardar
Apertura	(	[(← tope	Guardar
Cierre	)	[← tope (se quitó la ()	✓ Coincide
Apertura	{	[{ ← tope	Guardar
Cierre	}	[← tope (se quitó { }	✓ Coincide
Cierre	]	vacía ← tope	✓ Coincide
Fin de cadena	—	Pila vacía	→ True ✓

Algoritmo paso a paso

Paso 1. Recorrer la expresión carácter por carácter de izquierda a derecha.

Paso 2. Ignorar números, operadores (+-*/), espacios y letras.

Paso 3. Si el carácter es apertura ([{ , añadirlo a la pila.

Paso 4. Si el carácter es cierre)] }:

4a. Si la pila está **vacía** → retornar **False** (cierre sin apertura).

4b. Sacar el tope de la pila y verificar que corresponda al cierre actual.

4c. Si no coinciden → retornar **False**.

Paso 5. Al terminar el recorrido:

Si la pila quedó **vacía** → **True** ✓

Si aún quedan símbolos en la pila → **False** ✗ (aperturas sin cerrar).

Pares de símbolos válidos

Apertura	Cierre	Relación
(	)	Paréntesis
[	]	Corchetes
{	}	Llaves

Ejemplo de error: cierre incorrecto

Posición	Carácter	Estado pila	Decisión
0	[	[← tope	Apertura → guardar
1	(	[(← tope	Apertura → guardar
2	)	[← tope	✓ cierra (
3	]	vacía	✗ ERROR: tope era (, no [

La expresión "[(2)]" es inválida porque al encontrar] el tope de la pila es (, lo que no coincide. → retorna **False**.

Complejidad del algoritmo

Tipo	Notación	Razón
Tiempo	O(n)	Se recorre la cadena una sola vez.
Espacio	O(n)	En el peor caso la pila almacena n/2 aperturas.

Código Resuelto en Python

ejercicio1.py — validación con estructura de Pila (list)

1. Docstring y función principal

```
"""Ejercicio 1: validar símbolos de agrupación en una expresión.

Soporta: (), [] y {}
"""

def validacion_expresion_matematica(expresion_matematica):
    """Retorna True si la expresión está balanceada; False si no."""
```

2. Definir conjuntos de aperturas, cierres y pares

```
simbolos_apertura = "([{"      # aperturas reconocidas
simbolos_cierre   = ")]}"      # cierres reconocidos
pares = {"(": "(", "]": "[", "}": "{"}
        # cada cierre conoce su apertura
pila = []                    # pila vacía al inicio
```

3. Recorrer la expresión carácter por carácter

```
for simbolo in expresion_matematica:
    # Paso 2: ignoramos números, operadores, espacios, letras
    if simbolo in simbolos_apertura:
        pila.append(simbolo)    # Paso 3: guardar apertura
```

4. Lógica al encontrar un cierre

```
elif simbolo in simbolos_cierre:
    if not pila:
        return False          # cierre sin apertura previa
    ultimo_abierto = pila.pop() # sacar tope
    if pares[simbolo] != ultimo_abierto:
        return False          # cierre no coincide con apertura
```

5. Verificación final y ejemplos de uso

[illegible]

Resumen de decisiones clave

Decisión	Por qué
<code>pila = []</code>	Lista de Python como pila (append/pop son $O(1)$).
<code>simbolos_apertura = '([{'</code>	Membresía en string es $O(k)$ pero $k=3$, constante.
<code>pares = {")": "("}</code>	Diccionario permite lookup $O(1)$ del par esperado.
<code>pila.pop()</code>	Extrae el último abierto sin recorrer toda la pila.
<code>return len(pila) == 0</code>	Garantiza que no queden aperturas sin cerrar.

Estructura de Datos: Pila (Stack) · Complejidad: $O(n)$ tiempo · $O(n)$ espacio